



UNIVERSITÉ PARIS CITÉ
ÉCOLE D'INGÉNIEURS DENIS DIDEROT



Prédictions via les Graphons dans les Réseaux Sociaux

Présenté par:

SAAD ABDELKHALID

ROBAI Oussama

OUASSIL Mohammed Nassr Allah

Introduction & Motivation

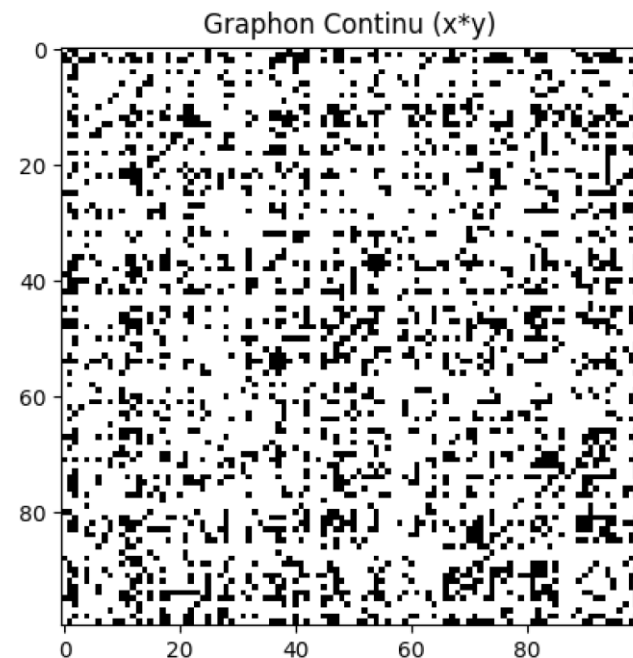
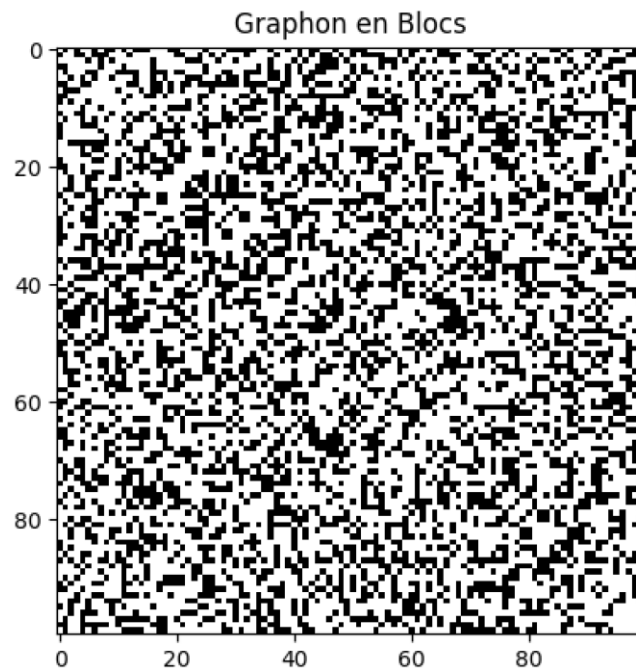
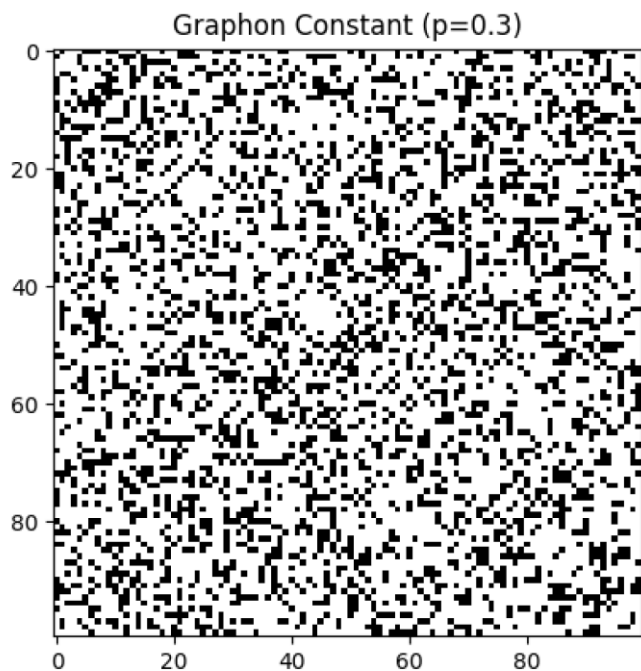
Introduction & Motivation

- Pourquoi modéliser la propagation d'informations/rumeurs ?
- Applications : réseaux sociaux, virus, marketing viral

**Qu'est-ce qu'un
graphon ?**

Qu'est-ce qu'un graphon ?

Visualisation des différents types de graphes et de graphons



Génération de réseaux

Génération de réseaux

Classe User:

```
class User:
    def __init__(self, id, age=None, city=None, interests=None, activity=1.0):
        self.id = id
        self.age = age if age is not None else np.random.randint(15, 55)
        self.city = city if city is not None else np.random.choice(['Paris', 'Lyon', 'Marseille'])
        self.interests = interests if interests is not None else np.random.uniform(0, 1, size=3)
        self.activity = activity
        self.friends = set()
    def add_friend(self, other):
        self.friends.add(other.id)
    def __repr__(self):
        return f"User(id={self.id}, age={self.age}, city={self.city}, act={self.activity:.2f})"

n = 40 # nombre d'utilisateurs
users = [User(i) for i in range(n)]
```

Génération de réseaux

Fonction Affinity :

La probabilité de lien entre deux individus x et y est donnée par :

$$W(x, y) = 0,15 \times \text{sim}(\mathbf{i}_x, \mathbf{i}_y) + 0,4 \times \frac{a_x + a_y}{2} + \text{bonus_âge}(g_x, g_y) + \text{bonus_ville}(v_x, v_y)$$

où :

— $\text{sim}(\mathbf{i}_x, \mathbf{i}_y)$: similarité cosinus entre les intérêts des deux individus ;

— a_x, a_y : niveaux d'activité des individus x et y ;

$$\text{— bonus_âge}(g_x, g_y) = \begin{cases} 0,2 & \text{si } |g_x - g_y| < 5 \\ 0 & \text{sinon} \end{cases} \quad \text{bonus_ville}(v_x, v_y) = \begin{cases} 0,3 & \text{si } v_x = v_y \\ 0 & \text{sinon} \end{cases}$$

Le tout est tronqué entre 0 et 1 :

$$W(x, y) = \min(\max(W(x, y), 0), 1)$$

Génération de réseaux

Fonction Affinity :

```
def affinity(u1, u2):  
    sim = np.dot(u1.interests, u2.interests) / (np.linalg.norm(u1.interests) * np.linalg.norm(u2.interests))  
    age_bonus = 0.2 if abs(u1.age - u2.age) < 5 else 0  
    city_bonus = 0.3 if u1.city == u2.city else 0  
    p = 0.15 * sim + 0.4 * (u1.activity + u2.activity)/2 + age_bonus + city_bonus  
    return min(max(p, 0), 1)
```

Génération de réseaux

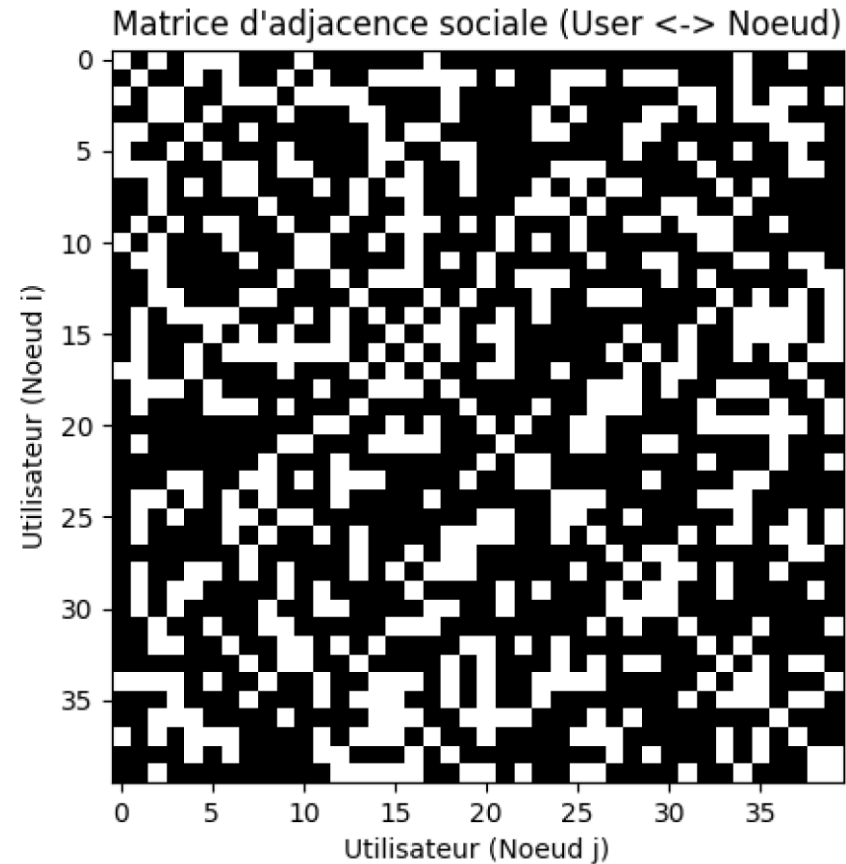
Matrice d'adjacense:

```
A = np.zeros((n, n), dtype=int)
for i in range(n):
    for j in range(i+1, n):
        if np.random.rand() < affinity(users[i], users[j]):
            A[i, j] = 1
            A[j, i] = 1
            users[i].add_friend(users[j])
            users[j].add_friend(users[i])
np.fill_diagonal(A, 0)

# 2. Affichage de la matrice d'adjacence
plt.figure(figsize=(5,5))
plt.imshow(A, cmap='Greys', interpolation='none')
plt.title("Matrice d'adjacence sociale (User <-> Noeud)")
plt.xlabel("Utilisateur (Noeud j)")
plt.ylabel("Utilisateur (Noeud i)")
plt.show()
```

Génération de réseaux

Matrice d'adjacense:



Simulation de propagation

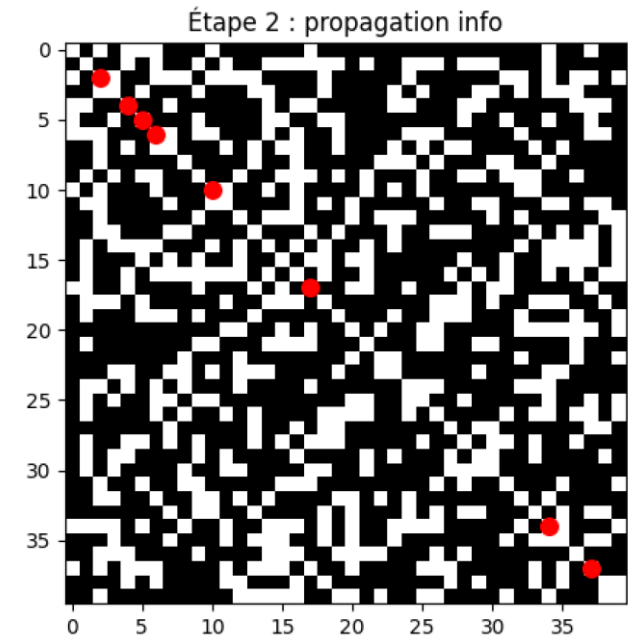
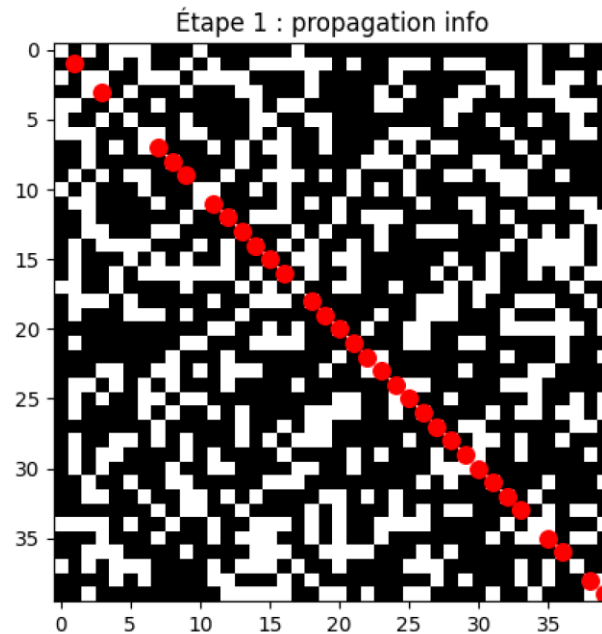
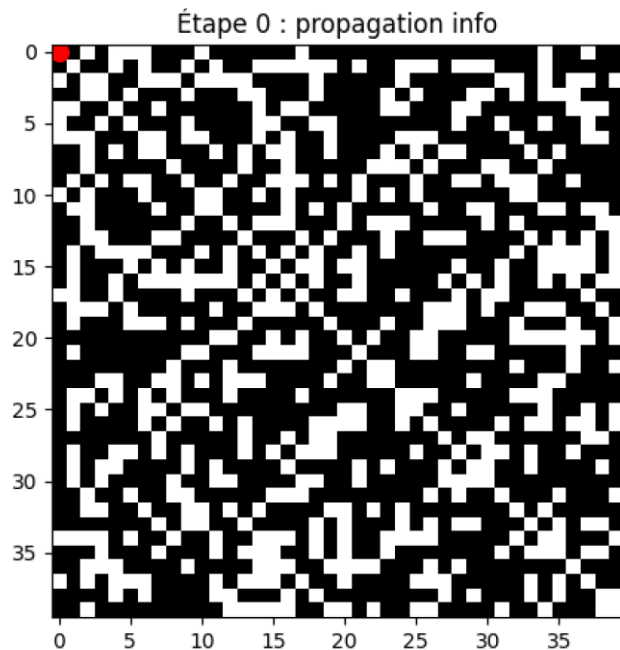
Simulation de propagation

On part d'un nœud, à chaque étape on propage à ses voisins non encore atteints

```
def simulate_propagation(A, seed=0, steps=8):
    n = A.shape[0]
    reached = set([seed])
    newly_reached = set([seed])
    propagation_steps = [set([seed])]
    for t in range(steps):
        next_new = set()
        for node in newly_reached:
            neighbors = set(np.where(A[node] == 1)[0])
            next_new |= (neighbors - reached)
        if not next_new:
            break
        newly_reached = next_new
        reached |= newly_reached
        propagation_steps.append(set(newly_reached))
    return propagation_steps
```

Simulation de propagation

On part d'un nœud, à chaque étape on propage à ses voisins non encore atteints



Simulation de propagation

On part d'un nœud, à chaque étape on propage à ses voisins non encore atteints

Départ : noeud 0

Étape 1 :

Nouveaux atteints : [1, 3, 7, 8, 9, 11, 12, 13, 14, 15, 16, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 35, 36, 38, 39]

Total atteints : 32

Liste totale : [0, 1, 3, 7, 8, 9, 11, 12, 13, 14, 15, 16, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 35, 36, 38, 39]

Étape 2 :

Nouveaux atteints : [2, 4, 5, 6, 10, 17, 34, 37]

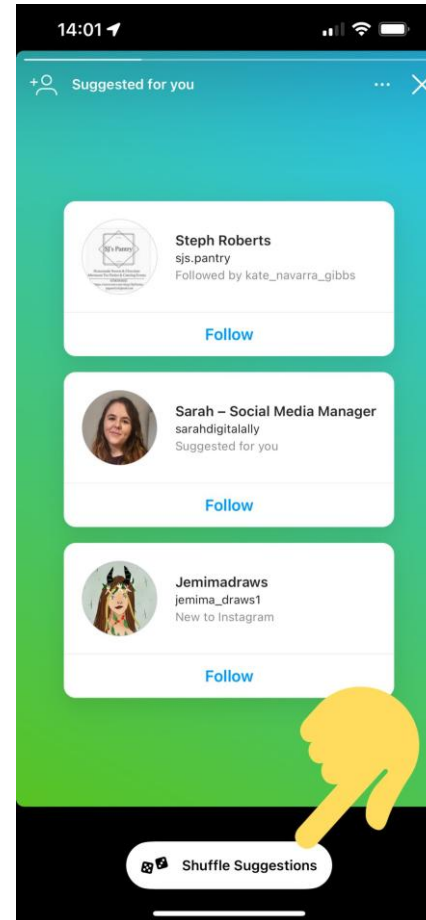
Total atteints : 40

Liste totale : [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39]

Étape 3 : plus aucun nouveau atteint, propagation terminée.

Analyses du réseau/ Suggestions d'amis

Suggestions d'amis



Suggestions d'amis

```
def get_suggestions(user_id, users, A, top_k=3):
    """Propose top_k suggestions pour user_id (non amis)"""
    n = len(users)
    friends = users[user_id].friends
    # Score par similarité d'intérêt (hors déjà amis et soi-même)
    suggestions = []
    for j in range(n):
        if j != user_id and j not in friends:
            sim = np.dot(users[user_id].interests, users[j].interests) / (
                np.linalg.norm(users[user_id].interests) * np.linalg.norm(users[j].interests)
            )
            common_friends = len(friends & users[j].friends)
            suggestions.append((j, sim, common_friends))
    suggestions.sort(key=lambda x: (x[2], x[1]), reverse=True) # tri sur amis communs puis similarité
    return suggestions[:top_k]
```

Suggestions d'amis

```
Suggestions pour User(id=0, age=28, city=Lyon, act=1.00) :  
- 18 | Marseille | 23 amis en commun | Score intérêts: 0.86  
- 23 | Marseille | 21 amis en commun | Score intérêts: 0.97  
- 31 | Paris | 20 amis en commun | Score intérêts: 0.87
```

**Pour aller plus loin :
GNN pour prédiction de
liens/rumeurs**

GNN pour prédiction de liens/rumeurs

```
class RumorSpreadGNN(nn.Module):
    def __init__(self, layers=5):
        super().__init__()
        self.layers = layers

    def forward(self, data):
        x = data.x.clone()
        edge_index = data.edge_index
        all_states = [x.clone()]
        for l in range(self.layers):
            x_new = x.clone()
            for i in range(x.shape[0]):
                neighbors = edge_index[1][edge_index[0]==i]
                if len(neighbors) > 0:
                    x_new[i] = torch.max(x[neighbors])
                if x[i] > 0:
                    x_new[i] = x[i]
            # Affichage étape par étape
            new_nodes = (x_new.view(-1) > 0) & (x.view(-1) == 0)
            print(f"\nÉtape {l+1}:")
            print("Nouveaux atteints :", torch.where(new_nodes)[0].numpy().tolist())
            print("Total atteints      :", int((x_new > 0).sum()))
            print("État :", x_new.view(-1).numpy())
            x = x_new
            all_states.append(x.clone())
        return x, all_states
```

GNN pour prédiction de liens/rumeurs

```
omna@DESKTOP-0R87QKV:/mnt/c/Users/nassr/OneDrive/Bureau/Graphon1/Rendu_Graphon1$ python3 gnn.py
```

```
Propagation terminée à l'étape 3 : plus aucun nouveau nœud atteint.
```

```
Étape 1:
```

```
Nouveaux atteints : [1, 2, 3, 6, 12, 15, 16, 17, 19, 20, 21, 22, 23, 27, 28, 29, 30, 31, 33, 34, 35, 37]
```

```
Total atteints : 23
```

```
Liste totale : [0, 1, 2, 3, 6, 12, 15, 16, 17, 19, 20, 21, 22, 23, 27, 28, 29, 30, 31, 33, 34, 35, 37]
```

```
Étape 2:
```

```
Nouveaux atteints : [4, 5, 7, 8, 9, 10, 11, 13, 14, 18, 24, 25, 26, 32, 36, 38, 39]
```

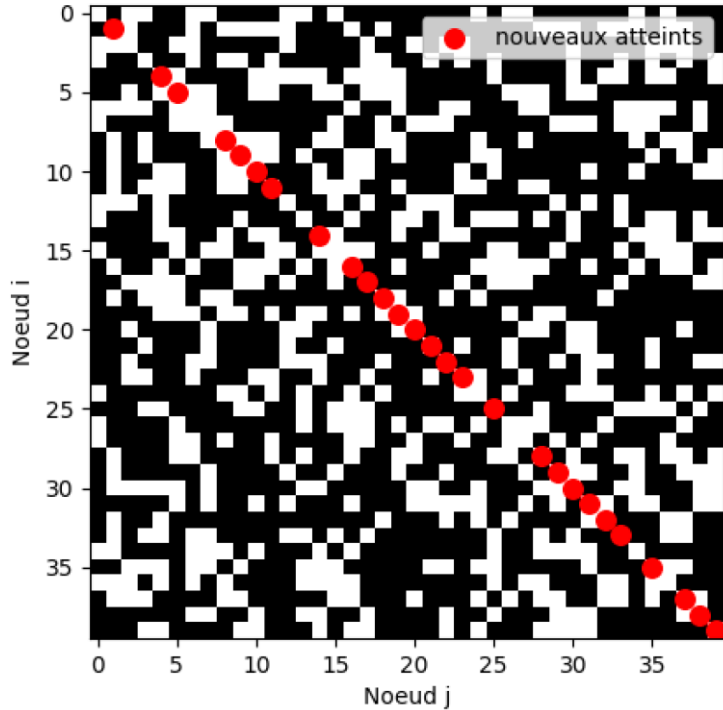
```
Total atteints : 40
```

```
Liste totale : [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39]
```

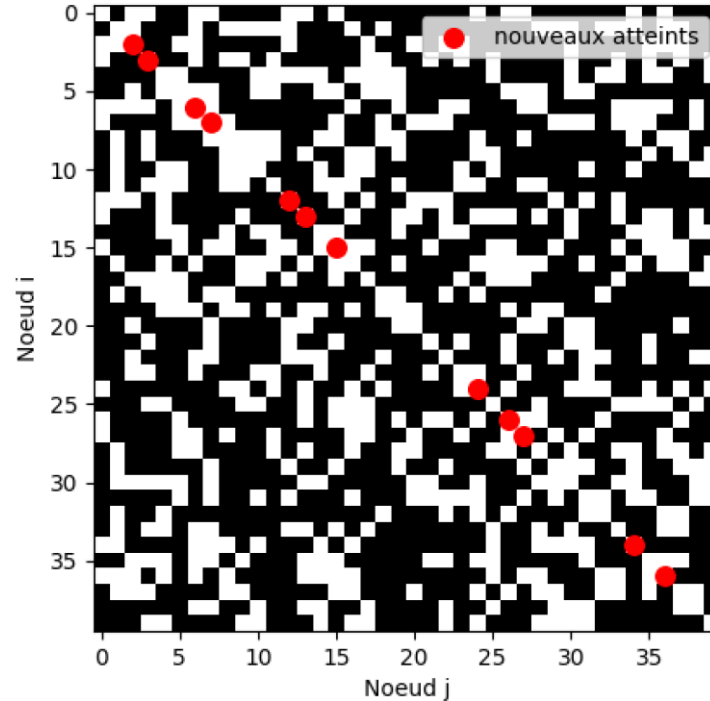
```
Étape 3: plus aucun nouveau atteint, propagation terminée.
```

GNN pour prédiction de liens/rumeurs

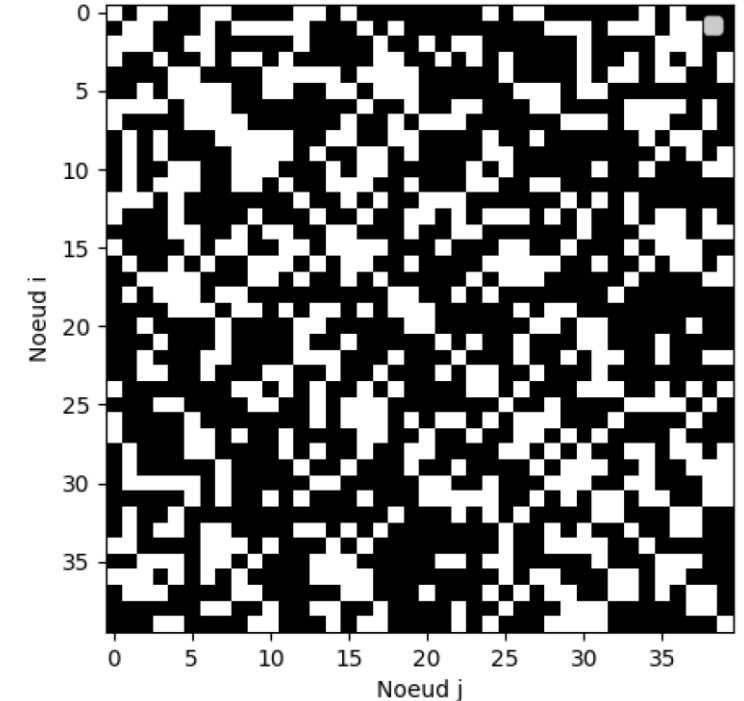
Étape 1 : nouveaux atteints (GNN)



Étape 2 : nouveaux atteints (GNN)



Étape 3 : nouveaux atteints (GNN)



Backtesting de validation: Vraie Data

Backtesting de validation : Vraie Data

Résumé des données Cora:

	Nombre de nœuds	Nombre d'arêtes	...	Nœuds en validation	Nœuds en test
0	2708	10556	...	500	1000

[1 rows x 7 columns]

=== VALIDATION ===

	precision	recall	f1-score	support
0	0.655	0.623	0.639	61
1	0.711	0.889	0.790	36
2	0.844	0.974	0.905	78
3	0.848	0.709	0.772	158
4	0.717	0.815	0.763	81
5	0.870	0.702	0.777	57
6	0.595	0.759	0.667	29
accuracy			0.772	500
macro avg	0.749	0.781	0.759	500
weighted avg	0.781	0.772	0.771	500

=== TEST FINAL ===

	precision	recall	f1-score	support
2	0.844	0.974	0.905	78
3	0.848	0.709	0.772	158
4	0.717	0.815	0.763	81
5	0.870	0.702	0.777	57
6	0.595	0.759	0.667	29
accuracy			0.772	500
macro avg	0.749	0.781	0.759	500
weighted avg	0.781	0.772	0.771	500

Conclusion

**Merci pour votre
attention**



UNIVERSITÉ PARIS CITÉ
ÉCOLE D'INGÉNIEURS DENIS DIDEROT



Prédictions via les Graphons dans les Réseaux Sociaux

Présenté par:

ROBAII Oussama

OUASSIL Mohammed Nassr Allah